the space. Notice how some regions are still only covered by nodes at level 1, because these intermediate nodes only have one child.

9.2.1 Constructing the BST

So far, we have just hinted at the construction of a binary search tree, implying that there is a direct translation of the partitions, or rather the pivots we choose, into a *BST*.

We have also implied that the BST we are going to construct is an important part of the k-d tree. Let's give a more formal definition to clarify the relation between these two data structures.

A k-d tree is a binary search tree whose elements are points taken from a k-dimensional space, i.e. tuples with k elements, whose coordinates can be compared (to simplify, let's assume each coordinate's values can be translated into real numbers). In addition to that, in a k-d tree, at each level `i` we only compare the `i`-th (modulo k) coordinate of points, to decide which branch of the tree will be traversed.

In fewer words: a k-d tree can be described in terms of a *binary search tree* with a fancy comparison method on its keys. The added value is given by the algorithms for search that can be performed on this kind of tree much more efficiently than on other, simpler, data structures.